Họ và tên: Nguyễn Hữu Hạo

MSSV: 2474802010108

BÁO CÁO THỰC HÀNH LAB 6

QUY HOẠCH TUYẾN TÍNH VÀ TỐI ƯU CÓ RÀNG BUỘC

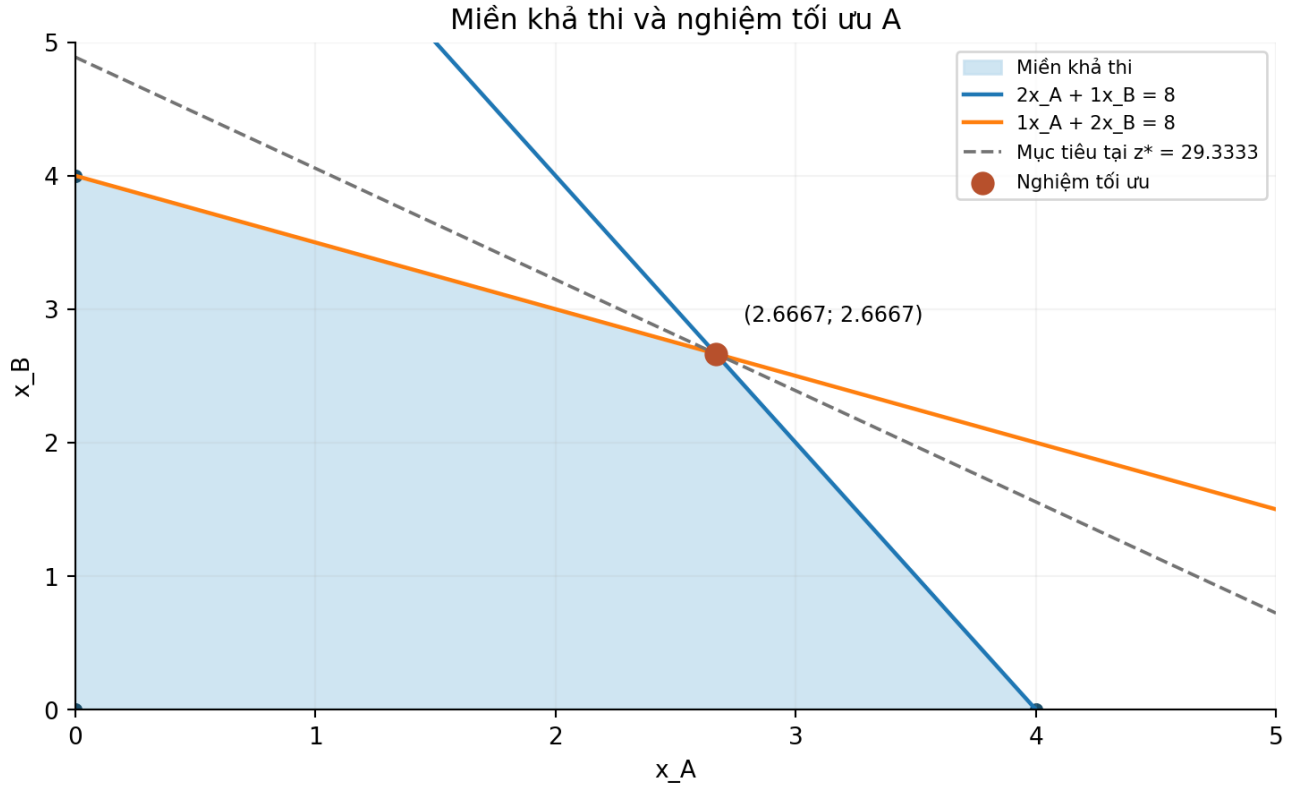
Báo cáo giải Bài toán A, B bằng SciPy và CVXPY, kiểm tra slack và active constraints, sau đó hoàn thành cả bốn bài tập. Hai solver cho nghiệm khớp nhau trong sai số 10^{-6}. Ở Bài toán B, tăng RAM thêm 20% tạo mức cải thiện objective lớn nhất trong ba thử nghiệm.

1 Mô hình Bài toán A

Biến quyết định xA và xB là mức phân bổ hai workload A và B, nhận giá trị thực không âm. Mỗi đơn vị A tạo 5 điểm hiệu quả; mỗi B tạo 6 điểm. Hai ràng buộc giới hạn hai nhóm tài nguyên.

$$max\ z\ =\ 5x_A\ +\ 6x_B$$

$$2x_A\ +\ x_B\ \leq\ 8;\quad x_A\ +\ 2x_B\ \leq\ 8;\quad x_A, x_B\ \geq\ 0$$



Hình 1 Miền khả thi và nghiệm tối ưu của hai workload A B

Nhận xét: Miền khả thi có các đỉnh (0;0), (4;0), (8/3;8/3), (0;4). Giá trị objective tại các đỉnh lần lượt là 0; 20; 29,3333; 24. Đỉnh giao hai đường tài nguyên cho giá trị lớn nhất; đây là nghiệm tối ưu liên tục.

Môi trường chạy: Python 3.12.14, SciPy 1.18.1, CVXPY 1.9.3, NumPy 2.5.4. SciPy dùng HiGHS; CVXPY dùng CLARABEL. Kiểm tra active khi |slack| ≤ 10^{-6}.

2 Giải Bài toán A và phân tích active constraints

Code chính bằng SciPy

```
c = np.array([-5, -6])  # linprog thực hiện MIN
A = np.array([[2, 1], [1, 2]])
b = np.array([8, 8])
result = linprog(c, A_ub=A, b_ub=b,
             bounds=[(0, None)]*2, method='highs')
x_star = result.x
z_star = -result.fun
slack = b - A @ x_star
```

Code kiểm tra chéo bằng CVXPY

```
x = cp.Variable(2, nonneg=True)
problem = cp.Problem(cp.Maximize(5*x[0] + 6*x[1]),
                 [2*x[0]+x[1] <= 8, x[0]+2*x[1] <= 8])
problem.solve(solver='CLARABEL')
```

Kết quả hai solver cho Bài toán A

```
SciPy HiGHS: success = True
x* = (2.66666667; 2.66666667)
z* = 29.33333333
slack = (0.00000000; 0.00000000)

CVXPY CLARABEL: status = optimal
x* = (2.66666667; 2.66666667)
z* = 29.33333333
```

Hình 2 Nghiệm tối ưu và trạng thái solver của Bài toán A

Bảng 1 Tự kiểm tra tài nguyên tại nghiệm A

Constraint	Used	Limit	Slack	Active
Tài nguyên 1	8.0000	8.0000	0.0000	Có
Tài nguyên 2	8.0000	8.0000	0.0000	Có

Nhận xét: Cả hai nhóm tài nguyên đều dùng hết 8 đơn vị nên đều active. Đổi dấu hệ số objective để SciPy tìm min của −z; khi đọc kết quả cần lấy −result.fun. Hai biến đều dương nên cận dưới không âm không active tại nghiệm này.

Câu hỏi tăng Constraint 1 thêm 1 đơn vị: giải lại với giới hạn (9;8) được x* = (3,3333;2,3333), z* = 30,6667, tăng 1,3333 so với ban đầu. Constraint 1 active gợi ý tài nguyên có thể đang gây nghẽn; việc giải lại xác nhận mức cải thiện trong trường hợp cụ thể này. Active đơn thuần không luôn bảo đảm mức tăng dương ở mọi mô hình khác.

3 Bài toán B phân bổ ba workload Cloud

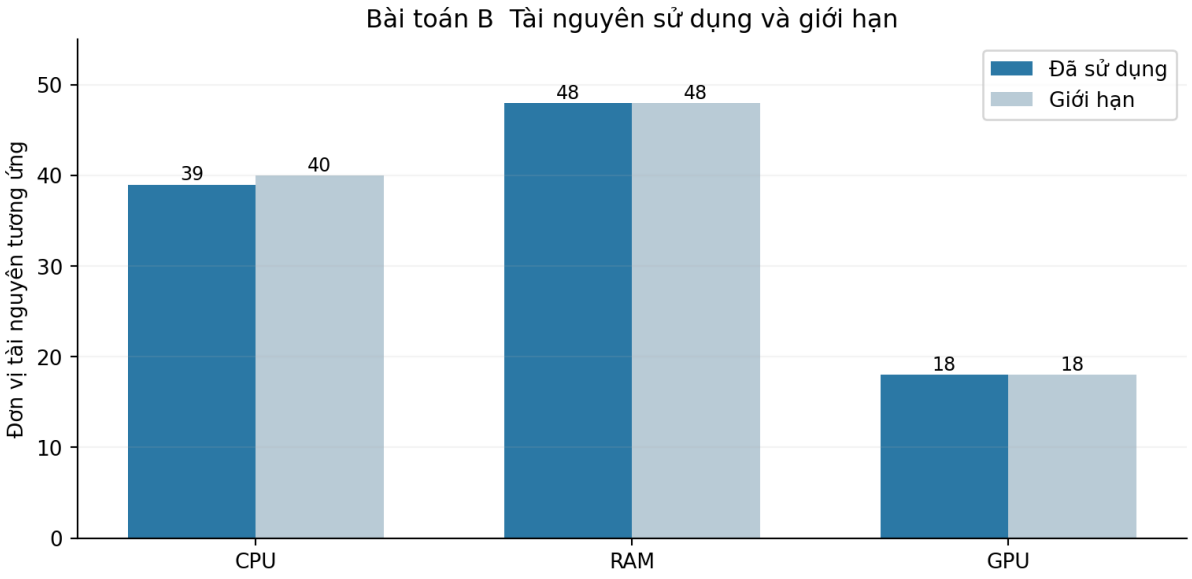
Biến x1, x2, x3 lần lượt là mức workload Inference, Training và Data Processing. Các biến là mức workload liên tục không âm; CPU, RAM và GPU dùng đơn vị tài nguyên theo đề.

$$max\ z = 8x_1 + 11x_2 + 6x_3$$
$$2x_1 + 4x_2 + x_3 \leq 40 \qquad (CPU)$$
$$4x_1 + 3x_2 + 2x_3 \leq 48 \qquad (RAM)$$
$$x_1 + 3x_2 \leq 18; \qquad x_1, x_2, x_3 \geq 0$$

Kết quả hai solver cho Bài toán B

```
SciPy: x* = (0.000000; 6.000000; 15.000000)
Objective = 156.000000
Slack CPU, RAM, GPU = (1.000000; 0.000000; 0.000000)
CVXPY: x* = (0.000000; 6.000000; 15.000000)
Objective = 156.000000
Status = optimal
```

Hình 3 Kiểm tra chéo nghiệm B bằng SciPy và CVXPY



Hình 4 Mức sử dụng và giới hạn từng tài nguyên tại nghiệm B

Bảng 2 Slack và active constraints của B

Tài nguyên	Used	Limit	Slack	Active
CPU	39.0000	40.0000	1.0000	Không
RAM	48.0000	48.0000	0.0000	Có
GPU	18.0000	18.0000	0.0000	Có

Nhận xét: Nghiệm x* = (0;6;15), z* = 156. CPU còn dư 1 đơn vị, trong khi RAM và GPU dùng hết. Ngoài hai ràng buộc tài nguyên RAM/GPU, cận x1 ≥ 0 cũng active vì x1 = 0. Mô hình chọn không phân bổ Inference do các hệ số lợi ích và giới hạn đang xét; nếu nghiệp vụ yêu cầu mức tối thiểu cần thêm ràng buộc tương ứng.

4 Bảng kiểm tra chéo và bài toán job nguyên

Bảng 3 Bảng báo cáo kết quả theo mẫu Mục 3.8

Bài toán	Solver	Nghiệm tối ưu	Objective	Active tài nguyên	Nhận xét
A	SciPy	(2.6667; 2.6667)	29.3333	TN1, TN2	Cả hai dùng hết.
A	CVXPY	(2.6667; 2.6667)	29.3333	TN1, TN2	Khớp SciPy.
B	SciPy	(0.0000; 6.0000; 15.0000)	156.0000	RAM, GPU	CPU dư 1.
B	CVXPY	(0.0000; 6.0000; 15.0000)	156.0000	RAM, GPU	Khớp SciPy.

Sai lệch max |Δx| giữa hai solver: A = 3.54e-11; B = 4.47e-09. Sai lệch objective cũng nhỏ hơn 10^{-6}. Các chênh lệch rất nhỏ do solver số học, không làm thay đổi kết luận về tài nguyên active.

Mục 3.9 và Bài 3 Nghiệm nguyên

Nếu xA, xB là số job không thể chia nhỏ, nghiệm (8/3;8/3) không triển khai trực tiếp. Liệt kê toàn bộ cặp trong 0 đến 8 và giữ những cặp thỏa hai bất đẳng thức.

Bảng 4 Nghiệm nguyên lân cận và ảnh hưởng của làm tròn

Cách chọn	(xA; xB)	Objective	Slack TN1 TN2	Khả thi
Làm tròn xuống	(2;2)	22	(2;2)	Có
Làm tròn gần nhất	(3;3)	33	(−1;−1)	Không
Lân cận	(3;2)	27	(0;1)	Có
Brute force tốt nhất	(2;3)	28	(1;0)	Có

Kết quả brute force bài toán job nguyên

```
So diem nguyen kha thi da kiem tra: 17
Best integer solution: [2, 3]
Best integer objective: 28
LP continuous objective: 29.33333333
Integrality gap: 4.545455%
```

Hình 5 Kết quả tìm nghiệm nguyên tối ưu bằng brute force

$$Gap = (z_{LP} - z_{integer}) / z_{LP} \times 100\% = 4.5455\%$$

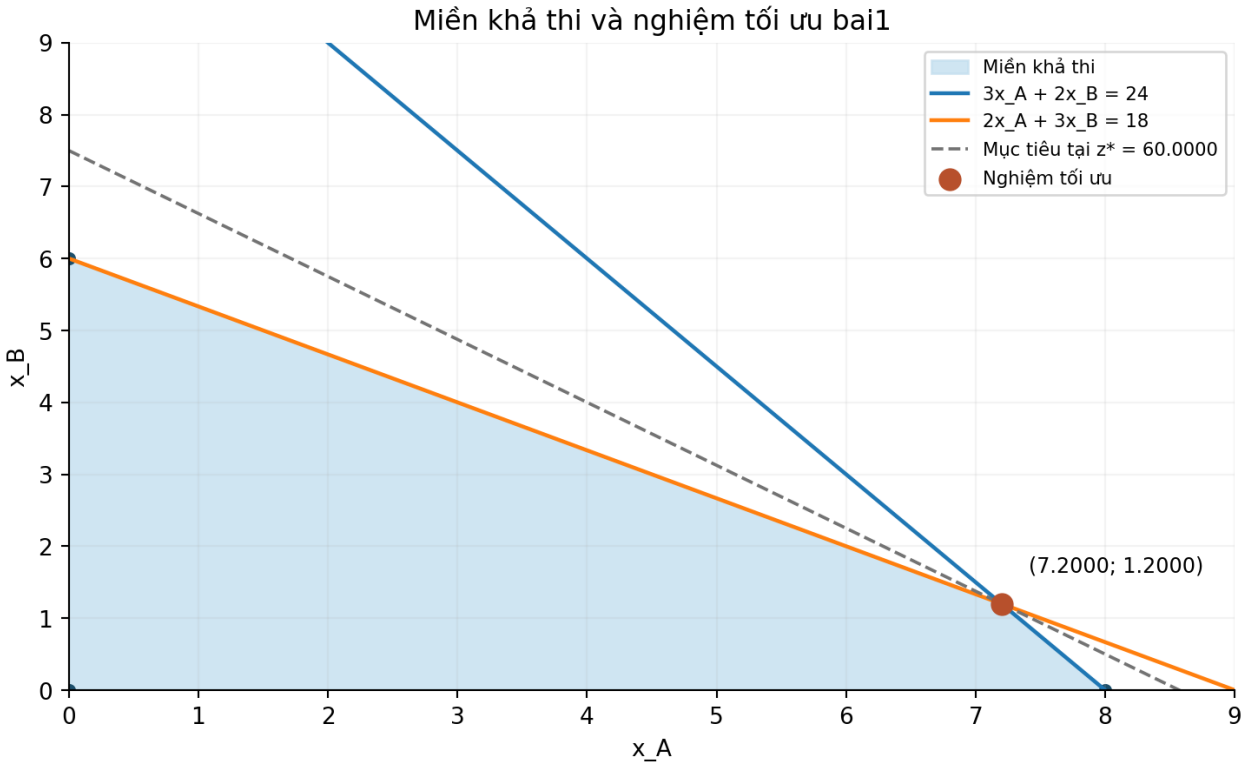
LP liên tục đủ dùng khi biến là mức tài nguyên có thể chia nhỏ hoặc dùng để đánh giá mức lợi ích chặn trên. Khi biến là số job/server/GPU vật lý cần mô hình nguyên. Trong bài này làm tròn xuống vẫn khả thi nhưng objective chỉ 22, thấp hơn 28. Làm tròn gần nhất thành (3;3) vi phạm cả hai giới hạn. Vì vậy phải kiểm tra feasibility và tối ưu lại trên biến nguyên thay vì làm tròn máy móc.

5 Bài 1 Phân bổ dịch vụ với 24 CPU và 18 RAM

Biến quyết định xA, xB là mức dịch vụ A, B không âm. A dùng 3 CPU và 2 GB RAM, tạo 7 điểm; B dùng 2 CPU và 3 GB RAM, tạo 8 điểm.

$$max\ z\ =\ 7x_A\ +\ 8x_B$$

$$3x_A\ +\ 2x_B\ \leq\ 24;\quad 2x_A\ +\ 3x_B\ \leq\ 18;\quad x_A, x_B\ \geq\ 0$$



Hình 6 Miền khả thi của bài phân bổ 24 CPU và 18 GB RAM

Bảng 5 Nghiệm tối ưu của Bài 1

Solver	xA	xB	Objective	Trạng thái
SciPy HiGHS	7.2000	1.2000	60.0000	Optimal
CVXPY CLARABEL	7.2000	1.2000	60.0000	optimal

Bảng 6 Kiểm tra slack của Bài 1

Constraint	Used	Limit	Slack	Active
CPU	24	24	0	Có
RAM	18	18	0	Có

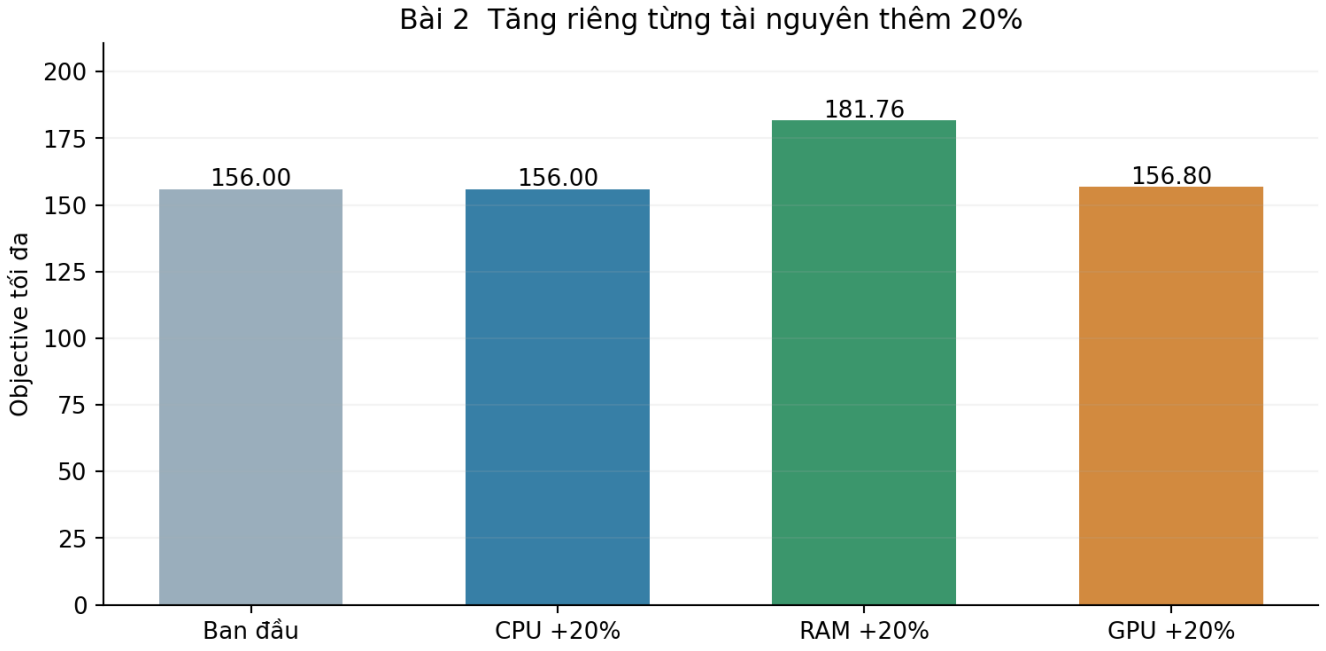
Nhận xét: Nghiệm liên tục là (7,2;1,2), objective 60. Cả CPU và RAM đều active; so với các đỉnh (8;0) cho 56 điểm và (0;6) cho 48 điểm, giao điểm hai đường tài nguyên tạo hiệu quả lớn hơn. Nếu dịch vụ được hiểu là số máy nguyên, cần thêm yêu cầu nguyên; đề hiện tại cho phép mức phân bổ liên tục.

6 Bài 2 Tăng riêng tài nguyên thêm 20 phần trăm

Dùng Bài toán B. Mỗi lần chỉ tăng một loại tài nguyên: CPU 40 → 48, RAM 48 → 57,6 hoặc GPU 18 → 21,6; hai giới hạn còn lại giữ nguyên.

Bảng 7 So sánh objective trước và sau tăng năng lực

Trường hợp	Nghiệm tối ưu	Objective	Δ Objective	Tăng %	Slack CPU RAM GPU
Ban đầu	(0.00; 6.00; 15.00)	156.00	0.00	0.00%	(1.00; 0.00; 0.00)
CPU +20%	(0.00; 6.00; 15.00)	156.00	0.00	0.00%	(9.00; 0.00; 0.00)
RAM +20%	(0.00; 4.48; 22.08)	181.76	25.76	16.51%	(0.00; 0.00; 4.56)
GPU +20%	(0.00; 6.40; 14.40)	156.80	0.80	0.51%	(0.00; 0.00; 2.40)



Hình 7 Objective khi tăng riêng CPU RAM hoặc GPU thêm 20 phần trăm

Bảng 8 Sự thay đổi nút thắt tài nguyên

Trường hợp	Active tài nguyên	Tài nguyên còn dư
CPU +20%	RAM, GPU	CPU dư 9.
RAM +20%	CPU, RAM	GPU dư 4,56.
GPU +20%	CPU, RAM	GPU dư 2,40.

Nhận xét: RAM +20% làm objective tăng 25,76 điểm, tương ứng 16,51%, lớn nhất trong ba thử nghiệm. GPU +20% chỉ tăng 0,80 điểm, khoảng 0,51%; CPU +20% không tăng objective. CPU ban đầu đã dư nên nới riêng CPU không giải quyết nút thắt RAM/GPU.

Sau khi tăng RAM hoặc GPU, CPU có thể trở thành active, làm lợi ích tăng thêm bị giới hạn. Ví dụ tăng GPU lên 21,6 nhưng nghiệm mới chỉ dùng 19,2 GPU, vì CPU và RAM đã hết. Slack ban đầu giúp nhận diện tài nguyên cần xem xét, nhưng mức lợi ích tăng 20% phải được xác nhận bằng giải lại mô hình.

7 Bài 4 Phân tích lỗi thiếu điều kiện không âm

Solver chỉ biết mô hình được khai báo. Nếu biến được cho phép nhận mọi giá trị thực, nghiệm có thành phần âm vẫn là nghiệm toán học hợp lệ nếu thỏa các ràng buộc đã viết. Giá trị âm không thể hiểu là số job hoặc mức tài nguyên âm trong bài phân bổ.

Ví dụ riêng để kiểm chứng lỗi bounds

$$max\ z = x_A + 2x_B; \quad x_A + x_B \leq 1; \quad x_B \leq 2$$

Ví dụ riêng minh họa lỗi thiếu điều kiện không âm

```
Mo hinh: max xA + 2*xB
xA + xB <= 1; xB <= 2

Free variables: x = [-1.0, 2.0], z = 3.0
Nonnegative:    x = [0.0, 1.0], z = 2.0
```

Hình 8 Nghiệm âm xuất hiện khi chủ động khai báo biến tự do

Nhận xét: Nếu cho biến tự do, solver chọn (−1;2), z = 3: lượng A âm bù cho B trong ràng buộc tổng tài nguyên. Khi thêm xA, xB ≥ 0, nghiệm có nghĩa là (0;1), z = 2. Đây là mô hình minh họa riêng, không phải sửa dữ liệu của Bài toán A hoặc B.

Cách khai báo và điểm khác nhau giữa hai thư viện

```
# SciPy mặc định bounds=(0, None), tức là đã có x >= 0.
free = linprog(c, A_ub=A, b_ub=b,
               bounds=[(None, None)]*2, method='highs')
nonneg = linprog(c, A_ub=A, b_ub=b, method='highs')
# CVXPY Variable mặc định là biến tự do; cần khai báo điều kiện.
x = cp.Variable(2, nonneg=True)
```

Với SciPy linprog, bỏ tham số bounds không tự làm biến nhận giá trị âm vì mặc định là không âm. Muốn biến tự do phải khai báo (None, None). Với CVXPY, cp.Variable mặc định không có cận dấu; cần nonneg=True hoặc thêm x ≥ 0. Đây là điểm cần kiểm tra khi chẩn đoán tình huống sinh viên trong đề.

Hai ví dụ AI và CNTT cần bounds

1. Phân bổ CPU/RAM cho container: mức phân bổ âm có thể bù giả cho container khác, làm tổng tài nguyên tính trên mô hình sai ý nghĩa thực tế.

2. Chia băng thông hoặc số lượt inference cho nhiều dịch vụ: lưu lượng/số job âm không triển khai được. Ngoài cận không âm có thể cần cận trên và điều kiện nguyên.

Kết luận

Mô hình A có nghiệm liên tục (8/3;8/3), objective 29,3333; nghiệm job nguyên tốt nhất là (2;3), objective 28 và gap 4,5455%. Mô hình B chọn (0;6;15), objective 156. Nếu ưu tiên mở rộng một tài nguyên thêm 20% trong phạm vi đã thử, RAM mang lại lợi ích lớn nhất. Các nghiệm đều được kiểm tra dấu, slack, trạng thái solver và đối chiếu SciPy/CVXPY.